

ADK (Agent Development Kit) 활용 연구

최초 작성일: 2026-05-19

업데이트/검증일: 2026-05-20

대상: LegalDocs Wise Pro 의 AI agent 파이프라인 (Stage 2~6)

주요 출처: [ADK 공식 문서](#), [TypeScript Quickstart](#), [TypeScript API Reference](#), [npm view @google/adk](#)

1. Executive Summary

ADK 는 Google 이 공개한 **production-grade agent 개발 프레임워크**다. 공식 문서 기준 Python / TypeScript / Go / Java / Kotlin 을 지원하며, 2026-05-20 현재 npm registry 의 `@@CODE0@@` 최신 버전은 `@@CODE1@@` 이다.

LegalDocs Wise Pro 는 이미 NestJS 위에 자체 agent 인프라를 상당히 구축해 둔 상태다. Gemini 직접 호출, stage 순차 흐름, prompt-studio 버전 관리, SSE 스트리밍, retry queue, Inspector 진단은 ADK 의 agent/workflow/context/evaluation/observability 영역과 자연스럽게 대응한다.

다만 이번 재검증 결과, 기존 문서의 핵심 전제 중 일부는 보정이 필요하다.

- **전면 교체는 여전히 비권장**. 한국 송무 도메인 로직(쟁점 트리, 증거 매칭, citation mapping, prompt-studio)을 ADK 추상화 위에 다시 얹는 비용이 크다.
- **부분 채택은 여전히 가치가 크다**. 특히 shadow 실행, golden-set 평가, context compaction, workflow orchestration 은 검토 가치가 있다.
- **TypeScript ADK 는 존재하지만, 모든 ADK 기능이 TS 동등 지원은 아니다**. `@@CODE0@@`, `@@CODE1@@`, `@@CODE2@@`, `@@CODE3@@` 등 agent 실행/워크플로우는 TS 로 가능하다. 반면 공식 문서상 `@@CODE4@@`, `@@CODE5@@`, `@@CODE6@@`, `built-in @@CODE7@@` 는 현재 Python/Java/Go/Kotlin 중심으로 표기되어 있다.
- **첫 PoC 는 "TS ADK wrapper + 자체 golden-set evaluator" 가 현실적이다**. ADK Python Evaluation 은 별도 harness 로 후속 검토한다.
- **런타임 선행 조건이 있다**. ADK TypeScript Quickstart 는 Node.js `@@CODE0@@`, `npm @@CODE1@@` 를 요구한다. 현재 로컬 확인값은 Node `@@CODE2@@`, `npm @@CODE3@@` 이므로, main backend 에 바로 의존성을 추가하기 전에 런타임 격리 또는 업그레이드가 필요하다.

핵심 판단:

판단	내용
전면 교체	🔴 비권장 — 도메인 로직 재이식 비용 과다
부분 채택	🟢 권장 — shadow 실행, 평가, context compaction, orchestration 중심
TypeScript 호환	🟡 부분 호환 — agent/workflow 는 가능, evaluation/trace/cache 는 언어별 지원 차이 확인 필요
첫 PoC	🟢 @@CODE0@@ 를 ADK @@CODE1@@ 로 shadow wrapping + 자체 golden-set 평가

2. ADK 개요 및 공식 지원 범위

2.1 정체성

ADK 의 방향성은 "프로토타입 agent" 가 아니라 운영 가능한 agent 시스템이다. 공식 문서는 agent 작성, multi-agent orchestration, graph workflow, 평가, 관측, 배포까지 하나의 프레임워크로 묶는 것을 강조한다.

TypeScript 최소 예시:

```
import { LlmAgent, GOOGLE_SEARCH } from '@google/adk';

const agent = new LlmAgent({
  name: 'researcher',
  model: 'gemini-flash-latest',
  instruction: 'You help users research topics thoroughly.',
  tools: [GOOGLE_SEARCH],
});
```

설치:

```
npm install @google/adk
npm install -D @google/adk-devtools
```

2.2 핵심 구성요소

영역	ADK 제공 기능
Agent	LLM agents, workflow agents, custom agents, multi-agent systems, agent routing
Workflow	Sequential / Loop / Parallel template workflow, ADK 2.0 graph/dynamic workflow
모델	Gemini, Gemma, Claude, Agent Platform hosted, Apigee AI Gateway, model routing, Ollama, vLLM, LiteLLM
도구	Function tools, MCP tools, OpenAPI tools, authentication, action confirmation
컨텍스트	Sessions, state, events, memory, context caching, context compression/compaction
실행	CLI, Web UI, API server, runtime config, event loop, resume/cancel

영역	ADK 제공 기능
평가	Criteria, user simulation, environment simulation, custom metrics, optimization
운영	Logging, metrics, traces, OpenTelemetry/Cloud Trace 계열 export
배포	Agent Runtime, Cloud Run, GKE

2.3 2026-05-20 기준 지원 범위 보정

아래 표가 이 문서의 핵심 업데이트다. ADK 는 다언어 프레임워크지만, 기능별 지원 언어가 균등하지 않다.

기능	공식 문서상 지원 상태	LegalDocs 판단
<code>@google/adk</code> TS agent 작성	TS Quickstart 제공. @@CODE0@@, @@CODE1@@ 예시 존재	🟢 NestJS shadow wrapper PoC 가능
TypeScript 런타임 요구	Node.js @@CODE0@@, npm @@CODE1@@	🟡 현재 로컬 Node/npm 과 불일치. PoC 격리 또는 업그레이드 필요
TS API	@@CODE0@@, @@CODE1@@, @@CODE2@@, @@CODE3@@, session/artifact/tool 관련 class 존재	🟢 실행 layer 검토 가능
Sequential workflow	Python v0.1.0 / TypeScript v0.2.0 / Go v0.1.0 / Java v0.2.0	🟢 단순 고정 stage pipeline 에 적합
Parallel workflow	Python v0.1.0 / TypeScript v0.2.0 / Go v0.1.0 / Java v0.2.0	🟢 Stage 3 분류/entity 병렬화 후보
Template workflow 상태	ADK 2.0 에서 graph/dynamic workflow 가 더 유연한 대안으로 소개됨	🟡 PoC 는 Sequential/Parallel, 장기 설계는 graph workflow 재검토
User Simulation	공식 문서상 ADK Python v1.18.0	🟡 TS mainline 에 바로 넣기보다 Python eval harness 로 분리
Custom Metrics	공식 문서상 ADK Python v1.18.0, Python 함수 signature	🟡 자체 TS metric 먼저. ADK custom metric 은 후속 Python harness
Traces	공식 문서상 Python v1.17.0 / Go v1.0.0 / Kotlin v0.1.0	🟡 TS built-in trace 전제 금지. NestJS OpenTelemetry 또는 ADK callback 기반 수동 계측
Context caching	공식 문서상 Python v1.15.0 / Java v0.1.0	🔴 TS PoC 범위에서 제외
Context compaction	공식 문서상 Python v1.16.0 / Java v0.2.0 / TypeScript v0.6.0	🟢 Stage 4/5 대용량 context 절감 후보

3. 현재 구현 분석 (LegalDocs Wise Pro)

3.1 스택

- **BE:** NestJS 11 + `@google/generative-ai` 직접 호출
- **FE:** Next.js 16. Stage orchestration 일부를 FE 가 주도
- **DB:** PostgreSQL + Prisma
- **Prompt 관리:** `@@CODE0@@` 정본 + Prisma `@@CODE1@@` / `PromptVersion` + Prompt Studio UI
- **현재 로컬 런타임 확인값:** Node `@@CODE0@@`, npm `@@CODE1@@`

3.2 Agent 파이프라인

file-router	파일명에서 핵심 문서 선별
stage2-extractor	사건기본정보 + 당사자 추출
stage3-classifier	자료 유형 분류
stage3-entity	Entity Dictionary 구축
stage4-analyzer	쟁점 트리 + 지식 그래프 (SSE 스트리밍)
stage5-strategy	소송 전략 / 추천 액션
stage6-chat	전략 채팅 + 서면 작성
stage6-drafting	서면 5분할 스트리밍 + 목차

핵심 구현 위치:

- `apps/backend/src/gemini/gemini.service.ts`
 - `generateJson<T>()`
 - `generateJsonStream()`
 - 수작업 timeout wrapper
- `apps/backend/src/cases/stage4-agent.service.ts`
 - Stage 4 background execution
 - SSE stream subject 관리
 - `MAX_RETRY = 1`
 - `mapAgentResponseToReplacePayload()` 로 agent response 를 정규화 snapshot 으로 변환
- `apps/backend/src/prompt-studio/prompt-loader.service.ts`
 - DB-first prompt loading
 - fallback prompt
 - usage stats

3.3 이미 갖춘 자체 agent 인프라

기능	현재 구현
Prompt 버전 관리	Prompt Studio + DB <code>PromptVersion</code> + seed
Prompt fallback	DB miss 또는 inactive 시 fallback const 사용
Prompt usage 진단	<code>PromptLoaderService.getUsageStats()</code> + Inspector
Structured output	Gemini <code>responseSchema</code> 직접 지정
Streaming	Stage 4 SSE, Stage 6 drafting stream
Retry / 내구성	Backend Stage 4 retry, frontend <code>be-sync-retry-queue</code>
상태 영속	Prisma(Stage 4), IndexedDB(Stage 2/3/5), freshness key
진단	Case Data Inspector, prompt usage, retry queue visibility
회귀 차단	Jest tests, seed/runtime key guard

결론: LegalDocs 는 이미 ADK 일부 기능을 도메인 특화 형태로 자체 구현했다. 따라서 ADK 는 "대체 플랫폼" 보다 평가/관측/워크플로우 보강 레이어로 보는 편이 맞다.

4. ADK ↔ 현재 구현 매핑

ADK 기능	현재 구현	평가
<code>LlmAgent</code>	stage service 의 Gemini 직접 호출	⚖️ 기능상 동등. ADK wrapper 로 코드량 축소 가능
<code>FunctionTool</code> / MCP tools	일부 도구 호출은 앱 내부 service 로 직접 구현	🟡 장기적으로 agent tool 경계 정리 가능
Sequential workflow	FE/BE 에 stage 순서가 분산	🟢 BE orchestration 집중 후보
Parallel workflow	Stage 대부분 직렬	🟢 Stage 3 독립 작업 병렬화 후보
Graph workflow	없음	🟡 ADK 2.0 방향성. PoC 이후 장기 검토
Structured output	Gemini <code>responseSchema</code>	⚖️ 현재 구현 유지 가치 큼. ADK TS 의 schema parity 별도 검증 필요
Context compaction	없음. prompt 에 대형 JSON 직접 삽입	🟢 TS 지원 명시. Stage 4/5 후보
Context caching	없음	🔴 공식 문서상 TS 지원 표기 없음. 우선 제외
Sessions / Memory	Prisma + IndexedDB + freshness key	🟡 도메인 상태는 현재 구조 유지가 안전
Prompt versioning	Prompt Studio	🟢 현재 우위. ADK instruction inline 으로 대체하지 않음

ADK 기능	현재 구현	평가
Streaming	SSE 수작업	⚖️ 운영 UX 와 강결합. PoC 에서는 유지
Resume / Cancel	일부 retry/status 는 있음, 표준 resume/cancel 은 제한적	🟡 장시간 분석 UX 개선 후보
Evaluation	수동 browser QA 중심	🟢 가장 큰 공백. 단, ADK user simulation/custom metrics 는 Python 중심
Observability / Traces	console/Nest logger + Inspector	🟡 개선 필요. TS built-in ADK trace 전제는 금지
Model routing	model name string 주입	🟡 ADK abstraction 또는 별도 router 로 개선 가능

범례: 🟢 도입 가치 높음 / 🟡 조건부 검토 / ⚖️ 동등 또는 현행 유지 / 🔴 현재 PoC 에서 제외

5. 권장 활용 시나리오

5.1 시나리오 A — Shadow-first 점진 보강 (권장)

현재 운영 경로는 그대로 두고, ADK 를 실행/평가/관측 보조 레이어로만 붙인다.

권장 순서:

1. P0 — 런타임 격리 또는 업그레이드

- ADK TS 요구사항: Node @@CODE0@@, npm @@CODE1@@
- 현재 local: Node @@CODE0@@, npm @@CODE1@@
- main backend 에 바로 추가하지 말고 별도 PoC workspace 또는 toolchain 고정 후 진행

2. P1 — 자체 golden-set evaluator

- ADK Python Evaluation 을 바로 붙이지 않는다.
- 먼저 TypeScript 로 Stage 4/5 output JSON 을 비교하는 deterministic evaluator 를 만든다.
- 예: schema validity, issue label Jaccard, citation precision, evidence link match, recommended action match.

3. P1 — Stage 4 ADK LlmAgent shadow wrapper

- @@CODE0@@, @@CODE1@@ 는 현행을 그대로 사용.
- ADK 는 실행 wrapper 역할만 한다.
- 결과는 운영 DB 에 commit 하지 않고 golden-set report 로만 저장한다.

4. P1 — 관측 보강

- 공식 문서상 TS built-in trace 지원이 명시되지 않았으므로 ADK Trace 직접 도입을 전제하지 않는다.

- NestJS OpenTelemetry, Cloud Trace direct exporter, ADK callback/logging hook 조합을 검토한다.

5. P2 — Context compaction

- TS 지원이 공식 문서에 명시되어 있다.
- Stage 4/5 의 대응량 prompt context 에 대해 token/latency A/B 를 측정한다.

6. P2 — Sequential/Parallel workflow

- Stage 3 classifier/entity 처럼 독립성이 높은 작업부터 병렬화 후보로 본다.
- FE 단계별 UX, 사용자의 중간 개입, freshness/retry 정책과 충돌하지 않는 범위에서만 이관한다.

5.2 시나리오 B — Workflow orchestration 일부 이관

FE 가 분산 주도하는 stage 흐름을 BE 의 ADK workflow 로 선언하는 방안이다.

개념 예시:

```
import { LlmAgent, ParallelAgent, SequentialAgent } from '@google/adk';

const stage3Parallel = new ParallelAgent({
  name: 'stage3_parallel',
  subAgents: [stage3ClassifierAgent, stage3EntityAgent],
});

const intakePipeline = new SequentialAgent({
  name: 'case_intake_pipeline',
  subAgents: [
    fileRouterAgent,
    stage2ExtractorAgent,
    stage3Parallel,
    stage4AnalyzerAgent,
    stage5StrategyAgent,
  ],
});
```

주의점:

- ADK 문서상 `ParallelAgent` 는 독립 branch 로 실행되며 자동 state 공유가 제한적이다.
- 공유 state, DB write, race condition 은 직접 설계해야 한다.
- ADK 2.0 에서 template workflow 는 graph/dynamic workflow 로 대체되는 방향이므로 장기 설계는 graph workflow 를 다시 확인해야 한다.

5.3 시나리오 C — 전면 채택 (비권장)

상세 분석: [41-adk-full-migration-2026-05-20.md](#) — 전면 교체의 정의/로드맵/C-1.C-2 분기/
정당화 조건을 단독 문서로 다룬다.

모든 stage 를 ADK agent 로 재작성하고 Agent Runtime 중심으로 배포하는 방식이다.

비권장 이유:

- 한국 송무 도메인 schema 와 mapping 로직을 다시 이식해야 한다.
- Prompt Studio 와 ADK instruction 관리가 충돌할 수 있다.
- Stage 4 SSE UX, snapshot commit, retry/status 전이가 이미 운영 코드에 녹아 있다.
- ADK TS 기능 성숙도와 문서상 지원 범위가 Python 과 동일하지 않다.

6. 첫 PoC 제안 (수정안)

6.1 목표

Stage 4 analyzer 를 운영 경로와 분리된 shadow mode 로 실행하고, 기존 Gemini 직접 호출 대비 ADK wrapper 의 품질/지연/비용/스키마 안정성을 측정한다.

6.2 범위

포함:

- @google/adk@1.1.0 기반 TypeScript PoC
- Stage 4 prompt 재사용
- 운영 DB commit 없음
- golden-set 5건 이상
- 자체 TS metric report

제외:

- ADK Python `adk eval`
- User Simulation
- ADK Custom Metrics
- ADK built-in Trace
- Context caching
- 운영 Stage 4 endpoint 교체

6.3 구현 단계

1. PoC runtime 준비

- Node @@CODE0@@, npm @@CODE1@@ 확보
- main backend 와 충돌하지 않도록 별도 workspace 또는 script 로 격리

2. Stage 4 shadow runner 작성

- 현행 `stage4-v2` prompt builder 를 그대로 호출

- ADK `LlmAgent` 로 실행
- response 는 `Stage4V2AgentResponse` 로 parse/validate

3. Golden set 구성

- 입력: `@@CODE0@@`, `@@CODE1@@`, `@@CODE2@@`, `@@CODE3@@`
- 기대값: issue labels, party aliases, evidence links, citation targets, recommended usage strategy

4. Metric 정의

- `schema_validity` : JSON parse + required fields validation
- `issue_count_score` : 기대 쟁점 개수와 실제 개수의 근접도
- `issue_label_jaccard` : 쟁점명/핵심 키워드 set 유사도
- `citation_precision` : 기대 citation target 대비 precision
- `evidence_link_score` : evidence id/name match 점수
- `@@CODE0@@`, `@@CODE1@@`, `output_tokens` 가능 시 수집

5. Decision gate

- schema validity 100%
- 핵심 metric 이 현행 baseline 대비 동등 이상
- latency/cost 가 baseline 대비 10% 이상 악화되지 않을 것
- 운영 endpoint/DB write 영향 0

7. 적용 후보 우선순위

우 선	작업	기대 효과	규 모	비고
P0	Node/npm 런타임 격리 또는 업그레이드	ADK TS PoC 가능 조건 확보	소~중	현재 local runtime 미달
P1	자체 TS golden-set evaluator	prompt 회귀 정량화, 수동 QA 감소	중	ADK Python eval 전 단계
P1	Stage 4 ADK shadow wrapper	ADK 실행 layer 적합성 검증	중	운영 경로 무영향
P1	관측 보강(Nest OTel/Cloud Trace direct/callback)	stage 별 latency/token 추적	중	TS built-in ADK Trace 전제 금지
P2	Context compaction	대용량 context 비용/지연 절감	중	TS 지원 명시
P2	Sequential/Parallel workflow	BE orchestration 집중, 병렬화	대	FE UX/freshness 와 조율 필요

우 선	작업	기대 효과	규 모	비고
P2	ADK Python evaluation harness	User simulation/custom metric 활용	중~대	Python runtime 별도 도입
P3	Model routing	Gemini/Claude/fallback 실험	소~중	정책/비용 기준 필요
P3	Context caching	반복 대형 prompt 비용 절감	중	공식 TS 지원 전까지 보류
P3	Resume/Cancel 표준화	장시간 Stage 4 UX 개선	중	현행 status/retry 와 통합 필요

8. 리스크 / 트레이드오프

리스크	설명	완화
기능별 언어 지원 차이	ADK 는 다언어지만 evaluation/trace/cache 는 TS 동등 지원이 아님	기능별 지원 matrix 를 기준으로 PoC 범위 제한
런타임 불일치	ADK TS Quickstart 요구 Node/npm 이 현재 local 과 다름	별도 PoC workspace 또는 runtime upgrade
ESM/NestJS 통합 비용	ADK TS 예시는 ES module 기반. 현행 Nest 빌드와 충돌 가능	main backend 직접 통합 전 isolated script 검증
Structured output parity	현행은 Gemini responseSchema 에 의존. ADK wrapper 에서 동일 보장 여부 확인 필요	Stage 4 schema validity 를 decision gate 로 둬
Prompt Studio 충돌	ADK instruction inline 과 DB prompt versioning 이 중복될 수 있음	prompt source of truth 는 현행 Prompt Studio 유지
이중 관측 체계	Inspector, Nest logger, ADK event, OTeI 이 분산될 수 있음	trace/span naming convention 을 먼저 정의
Workflow race condition	ParallelAgent 는 자동 state 공유가 제한적	병렬 branch 는 독립 입력/출력 + post-processing 으로 제한
ADK 버전 churn	공식 문서는 ADK 2.0 을 강조하지만 npm TS latest 는 1.1.0	버전 pinning, release note 확인, PoC 를 작게 유지

9. 권고

1. 시나리오 A(Shadow-first 점진 보강)로 진행한다.
2. 첫 PoC 는 "Stage 4 ADK TS wrapper + 자체 golden-set evaluator" 로 수정한다.
3. ADK Python Evaluation 은 후속 검토로 둔다. User Simulation/Custom Metrics 는 공식 문서상 Python 중심이므로 main NestJS runtime 에 섞지 않는다.

4. **Cloud Trace** 는 **ADK TS built-in** 으로 전제하지 않는다. NestJS OpenTelemetry 또는 ADK callback 기반 수동 계측으로 먼저 접근한다.
5. **Context caching** 은 보류하고, **Context compaction** 부터 본다.
6. **Prompt Studio** 는 계속 **source of truth** 로 유지한다. ADK 는 prompt versioning 을 대체하지 않는다.

다음 액션:

- golden-set 5건 선정: Stage 4 쟁점 추출 3건, Stage 5 추천 2건
- PoC runtime 결정: Node/npm 업그레이드 또는 별도 workspace
- `@google/adk@1.1.0` pinning 후 Stage 4 shadow runner spike
- 자체 metric report JSON 포맷 정의
- 결과가 baseline 을 통과하면 ADK Python eval harness 도입 여부 재판단

10. 참고 링크 및 확인값

공식 문서:

- [ADK Home](#)
- [TypeScript Quickstart](#)
- [TypeScript API Reference](#)
- [Sequential workflow](#)
- [Parallel workflow](#)
- [User Simulation](#)
- [Custom Metrics](#)
- [Traces](#)
- [Context caching](#)
- [Context compaction](#)

로컬/registry 확인값:

```
date: 2026-05-20
npm view @google/adk version: 1.1.0
node -v: v23.11.1
npm -v: 10.9.2
```